

Business Model

Verixa, an agentic AI quality engineer that works alongside testers

DESIGNED FOR Verixa	DATE September 21, 2026
DESIGNED BY Arisha Mumtaz and Farheen Imam	VERSION 1.1

KEY PARTNERS ● GitHub: stores the code repositories and sends pull request changes and webhook events ● Atlassian Jira: the tool where approved defect tickets are filed ● Google: ADK framework that runs the agents and saves their progress ● Language model providers: a main model for hard tasks and a fast model for simple, high volume tasks and backup. Currently Gemini and Groq, which can be changed later ● Playwright and Newman: the tools that run the UI and API tests for Verixa ● Cloud and database hosts: places to run PostgreSQL with pgvector, the backend, and the staging app ● SZABIST University: academic supervision by Asim Riaz	KEY ACTIVITIES ● Agent pipeline: building and looking after the Manager, Analysis, Test, and Report agents, with human checkpoints at Run Now and Jira approval ● Risk and test execution: rule based risk labels, then running relevant Playwright and Newman tests and telling real bugs apart from temporary errors and features not yet deployed ● Test Library and coverage: reading uploaded tests and writing new tests for coverage gaps ● Evidence and reporting: collecting evidence, removing secrets from it, writing a root cause, and drafting the ticket ● Platform and integrations: keeping GitHub, Jira, Playwright, and Newman working, plus the backend and dashboard ● Semester two: QA Memory, Suggest a Fix, Regression Comparison, and Cross-PR Risk Trend Reporting	VALUE PROPOSITIONS ● An AI teammate for testers: Verixa assists the tester and does not replace them. As Claude Code helps developers, Verixa helps testers, with a human making the final call at the two key steps ● Testers: no more retyping risk scores, screenshots, and error messages across five separate tools, with one connected loop from pull request to drafted ticket ● Small teams with limited testing resources: consistent, evidence backed defect reports without a lot of manual work ● Larger QA departments: less time matching logs by hand, more time for exploratory testing and judgement calls ● Developers: a ticket that already carries the evidence and a plain language root cause ● Coverage: new test cases drafted for gaps in the uploaded tests ● Trust: two human checkpoints, secrets removed from evidence, and features not yet deployed never reported as failures ● 8th Semester: finding similar past bugs, suggested fixes for developer review, comparing test runs, and risk trends across pull requests	CUSTOMER RELATIONSHIPS ● Self service: connect GitHub and Jira through one login, upload tests, and start running with no manual webhook setup ● Human in the loop: the tester stays in charge of the two most important steps, running the tests and approving the ticket ● Personal assistance: a support channel for setup help or when a result turns out to be wrong ● Feedback loop: corrections to risk labels and root causes could be used to improve accuracy in future work ● Community: open documentation and a public roadmap	CUSTOMER SEGMENTS ● Testers and QA engineers: the day to day users who review, run, approve, and file ● Small software teams and startups: few testers or little testing time, need consistent coverage ● Growing engineering and QA teams: high pull request volume, want less manual evidence work ● Developers: receive the finished ticket with evidence and root cause ● QA leads and managers: want a project wide view of risky modules (semester two) ● Common to all: teams in IT and software development that review pull requests on GitHub and track defects in Jira
KEY RESOURCES ● Agent pipeline: the core technical asset, built on Google ADK ● Team: two people across agent logic, backend, and integrations, with a project supervisor ● AI access: Google ADK and free language model APIs, with the provider open to change ● Data layer: PostgreSQL with pgvector for state, risk scores, test runs, and Jira links ● Test Library: the Playwright tests and Postman collection uploaded by the tester ● Evidence store: screenshots, traces, and logs that Verixa collects when it runs the tests ● QA Memory data set: past bugs and root causes, growing once semester two ships	CHANNELS ● Direct website: sign up flow and dashboard for connecting projects and reviewing evidence ● Marketplaces: GitHub Marketplace and Atlassian Marketplace listings ● Community: talking with developer communities and word of mouth ● Documentation: open documentation and public roadmap			
COST STRUCTURE ● Hosting and storage: backend, PostgreSQL database, staging environment, and growing evidence files ● Language model usage: API calls to the chosen model providers, free at project scale and paid as volume grows ● Engineering time: keeping integrations working as GitHub, Jira, Playwright, and Newman change ● Support and community: documentation, setup help, and community support as the user base grows	REVENUE STREAMS ● Free for small startups: the whole product, including the semester two features, is free for small startups and small teams, within fair usage limits ● Donations and sponsorship: optional, to help cover hosting and language model costs as usage grows ● No locked features: no feature is kept behind a paid plan ● Optional paid support: only for larger teams with very high pull request volume, or if we later move to better paid language models to keep quality high, and never required for small startups			

Built from the scope, users, and technology in the Verixa Final Year Project proposal. Revenue, channels, and customer relationship approaches are proposed directions beyond the academic scope.